Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 24.12.2025.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaA_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**
9. **Svaki zadatak nosi po 8 bodova. Kod za testiranje algoritama u Main metodi donosi još jedan bod.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Insertion Sort*

Napisati ***Insertion Sort*** algoritam za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- Kreirati klasu ***InsertionSort*** i u okviru nje generisati metodu ***public static void sort(long[] arr)***

Zadatak 2 – *Kumulativna suma vrednosti levih suseda*

Napisati algoritam koji modifikuje listu tako što vrednost čvora zanemenjuje vrednošću sume svih elemenata koji se u listi nalaze sa njegove leve strane.

Dodatne napomene za rešavanje zadatka:

- Zahtevan potpis metode: ***public static void calculateCumulativeSum()***
- Metodu pisati u okviru klase ***LinkedList***

Primer:

- Input: ***LinkedList: 1 -> 0 -> 6 -> 8 -> 4***
- Output: ***0 -> 1 -> 1 -> 7 -> 15***

- Input: ***LinkedList: 1 -> 2 -> 4 -> 0 -> 6***
- Output: ***0 -> 1 -> 3 -> 7 -> 7***

Zadatak 3 – *Towers of Hanoi*

Hanojske kule su klasična matematička igra koja se sastoji od tri štapa i određenog broja diskova različitih veličina koji se mogu nizati na štapove. Igra počinje sa svim diskovima naslaganim na jednom štapu u rastućem redosledu (najmanji na vrhu), formirajući oblik konusa.

Problem: Cilj je prebaciti ceo set diskova sa početnog štapa (Izvor) na završni štap (Cilj), koristeći pomoćni štap (Pomoćni), poštujući sledeća pravila:

1. Samo jedan disk se može pomerati u datom trenutku.
2. Svaki pokret se sastoji od uzimanja gornjeg diska sa jednog od štapova i postavljanja na vrh drugog štapa.
3. **Zlatno pravilo:** Nijedan disk ne sme biti postavljen iznad diska koji je manji od njega.

<u>Dodatni zahtevi za rešavanje zadatka:</u>

1. Zadatak rešavati u okviru metode: ***public void SolveHanoi(int n, Stack source, Stack destination, Stack auxiliary)***
2. **Simulacija pomoću stekova:**
   - Svaki od tri štapa (Izvor, Cilj, Pomoćni) mora biti predstavljen objektom klase Stack.
   - Diskovi su predstavljeni celim brojevima (npr. disk veličine 3 je veći od diska veličine 1).
   - Operacije pomeranja diskova moraju se vršiti isključivo metodama ***push()*** i ***pop***().
3. **Rezultat izvršavanja:**
   - Metoda treba da ispisuje svaki korak (npr. **"Pomeren disk 1 sa štapa A na štap C"**).
   - Na kraju izvršavanja, source i auxiliary stekovi moraju ostati prazni, a destination stek mora sadržati sve diskove u ispravnom redosledu.

Primer:

Input:		[6, 5, 4, 3, 2, 1]	
Korak 0:	Source Stack	{6, 5, 4, 3, 2, 1}	1 je na vrhu steka
	Destination Stack	{ }	
	Auxiliary Stack	{ }	
Korak 1:	Source Stack	{6, 5, 4, 3, 2}	2 je na vrhu steka
	Destination Stack	{ }	
	Auxiliary Stack	{ 1 }	1 je na vrhu steka
Korak 2:	Source Stack	{6, 5, 4, 3}	3 je na vrhu steka
	Destination Stack	{ 2 }	2 je na vrhu steka
	Auxiliary Stack	{ 1 }	1 je na vrhu steka
Korak 3:	Source Stack	{6, 5, 4, 3}	3 je na vrhu steka
	Destination Stack	{ 2, 1}	1 je na vrhu steka
	Auxiliary Stack	{ }	
Korak n:	Source Stack	{ }	
	Destination Stack	{6, 5, 4, 3, 2, 1}	1 je na vrhu steka
	Auxiliary Stack	{ }	